Architecture for Fully Decentralized Hierarchical Graph-Based Search and Rescue for Multi-Agent Systems

Jash Shah

18 March, 2026

Contents

1 Purpose and Scope **4**

2 System Overview **4**

3 Environment and Agent Model **5**

4 Belief Representation and Update **5**
4.1 Purpose of the belief map . . . 5
4.2 Single-victim and multi-victim modes . . . 5
4.3 Initial belief . . . 5
4.4 Observation model . . . 6
4.5 Joint multi-agent update . . . 6
4.6 Numerical stabilization . . . 7
4.7 What happens after detection and rescue . . . 7

5 Why Shannon Entropy Is Used **7**

6 Option Set and Semantics **8**
6.1 `Explore` . . . 8
6.2 `Navigate` . . . 8
6.3 `Form` . . . 8

7 Option-Conditioned Potentials **8**
7.1 Exploration potential . . . 8
7.2 Navigation potential . . . 8
7.3 Formation potential . . . 8

8 Target Selection and Multi-Victim Allocation **9**
8.1 Single-target baseline . . . 9
8.2 Multi-victim target extraction . . . 9
8.3 Agent-to-target assignment . . . 9

9 Graph Construction Under Limited Communication **10**
9.1 Communication graph . . . 10
9.2 Node and edge features . . . 10
9.3 Worker message passing . . . 10

10 High-Level Manager **10**
10.1 When the manager fires . . . 10
10.2 Manager input embedding . . . 11
10.3 Variable-length sequence handling . . . 11
10.4 Manager output . . . 11
10.5 Why the manager does not use the full belief map at execution . . . 11

11 Low-Level Worker **11**
11.1 Worker input . . . 11
11.2 Worker output . . . 11

12 Initiation and Termination Sets **12**
12.1 Thresholds . . . 12
12.2 Conflict precedence . . . 12
12.3 `Explore` initiation and completion . . . 12
12.4 `Navigate` initiation and completion . . . 12
12.5 `Form` initiation and completion . . . 12

13 Physical Justification of the Formation Option **13**
13.1 Formation radius . . . 13
13.2 Which agent count should be used . . . 13

14 Termination Module **13**
14.1 Learned termination . . . 13
14.2 Termination triggers . . . 13
14.3 Conflict resolution when no alternative option is valid . . . 14

15 Training-Time Critics **14**
15.1 High-level critic . . . 14
15.2 Low-level critic . . . 14

16 Reward Design **14**
16.1 High-level reward . . . 14
16.2 Entropy exploit prevention . . . 15
16.3 Low-level worker reward . . . 15
16.4 Credit assignment-aware shared reward . . . 15
16.5 Option-conditioned dense shaping . . . 15

17 Mission-Specific Extension: Unknown Victim Count, Rescue Demand, and Parallel Subgroup Coordination **15**
17.1 Victim state representation . . . 16
17.2 Unknown victim count . . . 16
17.3 Victim discovery and reveal rule . . . 16
17.4 Victim-demand signal . . . 17

17.5 Parallel subgroup behavior . . . 17
17.6 Agent assignment to victims . . . 17
17.7 Rescue slot formation . . . 18
17.8 Formation initiation rule . . . 18
17.9 Formation completion and rescue completion . . . 19
17.10 Post-rescue option reassignment . . . 19
17.11 Mission-specific navigation potential . . . 19
17.12 Mission-specific formation potential . . . 20
17.13 Mission-specific low-level rewards . . . 20
17.14 Mission-specific high-level reward . . . 20
17.15 Execution summary for this mission . . . 20

18 Optimization and Learning **21**
18.1 High-level advantage and PPO . . . 21
18.2 Low-level advantage and MAPPO . . . 21
18.3 Why PPO for manager and MAPPO for worker . . . 21
18.4 Termination loss . . . 21

19 Replay / Rollout Storage Requirements **21**

20 Training Curriculum **22**
20.1 Phase 1: low-level pretraining . . . 22
20.2 Phase 2: termination warm-start . . . 22
20.3 Phase 3: joint training . . . 22
20.4 Phase 4: fine-tuning . . . 22

21 Execution Algorithm **23**

22 Decentralization Claim: What Is and Is Not Guaranteed **23**

23 Convergence and Stability Claim **23**

24 Scalability Claim **24**
24.1 Structural scalability . . . 24
24.2 Performance scalability . . . 24

25 Recommended Metrics **24**

26 Summary of Final Design Decisions **25**

1 Purpose and Scope

This document is a replacement-grade implementation specification for a hierarchical graph-based multi-agent search-and-rescue (SAR) framework. It is intended to supersede an earlier specification by resolving major implementation and presentation gaps while preserving the core design goals:

1. **Fully decentralized execution**: each agent acts using only its own local observation, local memory, current option, and messages from nearby neighbors.
2. **Scalable coordination under limited communication**: graph message passing is local and sparse.
3. **Hierarchical control**: a high-level manager selects temporally extended options and a low-level worker executes primitive actions.
4. **Uncertainty-aware exploration**: the search phase is driven by reduction of belief uncertainty using Shannon entropy.
5. **Trainable under CTDE**: centralized critics and centralized belief maintenance are used during training only.

This document is not written as a research paper. It is an implementation guide, mathematical reference, and system design specification.

2 System Overview

The SAR mission is divided into three behavior modes:

$$\mathcal{Z} = \{\texttt{Explore}, \texttt{Navigate}, \texttt{Form}\}.$$

The architecture has six functional components:

a) **Belief module**: maintains a centralized belief map over unresolved victim locations during training and evaluation.

b) **High-level manager**: chooses an option at option boundaries.

c) **Low-level worker**: produces primitive actions at every timestep.

d) **Local communication graph**: restricts information exchange to nearby agents.

e) **Termination module**: decides when the current option should stop.

f) **Centralized critics**: provide training-time value estimates at high and low levels.

At execution time, the runtime controller is decentralized:

$$z_k^i \sim \pi_H^i(\cdot \mid h_{H,k}^i), \qquad a_t^i \sim \pi_L^i(\cdot \mid h_{L,t}^i, z_k^i),$$

where agent i uses only local information and neighbor messages. The centralized belief map and centralized critics are *not required* at deployment.

3 Environment and Agent Model

Consider N agents in a discrete or continuous workspace with obstacle map $\mathcal{M}$ and a set of unresolved victims $\mathcal{V}_{\text{unres}}$.

At time t:

- agent i has position $p_t^i \in \mathbb{R}^2$,
- velocity or motion state v_t^i,
- local observation o_t^i,
- current active option z_k^i chosen at the most recent option boundary t_k.

The primitive action space may be discrete:

$$a_t^i \in \{\texttt{up}, \texttt{down}, \texttt{left}, \texttt{right}, \texttt{stay}\},$$

or continuous if the robot platform uses velocity commands.

4 Belief Representation and Update

4.1 Purpose of the belief map

The belief map is a training-time and evaluation-time mission state used to quantify search uncertainty and define option-conditioned macro rewards. It is maintained by the **central training server** (or simulator controller) and is not required as an explicit input to the decentralized worker at execution.

4.2 Single-victim and multi-victim modes

This specification supports two modes:

1. **Single-victim baseline mode**: one belief map $b_t(x)$ over cells $x \in \mathcal{X}$.
2. **Multi-victim mission mode**: a set of unresolved target hypotheses maintained from the same map, extracted as peaks/clusters from a shared victim-likelihood map.

The base belief map is

$$b_t(x) = P(\text{unresolved victim mass at cell } x \mid H_t),$$

where H_t is the accumulated training-side observation history.

4.3 Initial belief

Unless domain priors are available, use the uniform prior over all valid free cells:

$$b_0(x) = \frac{1}{|\mathcal{X}_{\text{free}}|}, \qquad x \in \mathcal{X}_{\text{free}}.$$

If prior information exists (e.g., likely victim zones, building semantics, thermal hotspots), initialize with:

$$b_0(x) \propto \pi_{\text{prior}}(x), \qquad \sum_x b_0(x) = 1.$$

4.4 Observation model

Each agent emits a binary victim-detection observation:

$$o_t^i \in \{0, 1\},$$

where $o_t^i = 1$ means detection and $o_t^i = 0$ means no detection.

Let:

- r_{det} = detection radius,
- $p_{fn}(d)$ = false negative rate as a function of distance d,
- $p_{fp}(d)$ = false positive rate as a function of distance d.

A simple piecewise-constant model is:

$$p_{fn}(d) = \begin{cases} p_{fn}, & d \le r_{\text{det}}, \\ 1, & d > r_{\text{det}}, \end{cases} \qquad p_{fp}(d) = \begin{cases} p_{fp}, & d \le r_{\text{det}}, \\ 0, & d > r_{\text{det}}. \end{cases}$$

For a hypothesized victim at cell x and agent position p_t^i, define $d_t^i(x) = \|p_t^i - x\|_2$. Then

$$P(o_t^i = 1 \mid x) = \begin{cases} 1 - p_{fn}, & d_t^i(x) \le r_{\text{det}}, \\ p_{fp}, & d_t^i(x) > r_{\text{det}}, \end{cases}$$

and

$$P(o_t^i = 0 \mid x) = 1 - P(o_t^i = 1 \mid x).$$

A smoother model may instead use a distance-dependent detection profile $q(d)$:

$$P(o_t^i = 1 \mid x) = q(d_t^i(x))(1 - p_{fn}) + (1 - q(d_t^i(x)))p_{fp},$$

with $q(d)$ monotonically decreasing.

4.5 Joint multi-agent update

To avoid order dependence, all agent observations at time t are treated as a joint observation under conditional independence given the victim hypothesis:

$$P(o_t^1, \ldots, o_t^N \mid x) = \prod_{i=1}^{N} P(o_t^i \mid x).$$

The Bayes update is:

$$b_{t+1}(x) = \frac{\left(\prod_{i=1}^{N} P(o_t^i \mid x)\right) b_t(x)}{\sum_{x' \in \mathcal{X}} \left(\prod_{i=1}^{N} P(o_t^i \mid x')\right) b_t(x')}.$$

This is the default implementation. Sequential per-agent updates are not recommended because overlapping observations make update order undesirable.

4.6 Numerical stabilization

To compute entropy safely, use:

$$\tilde{b}_t(x) = \max(b_t(x), \varepsilon_b), \qquad \bar{b}_t(x) = \frac{\tilde{b}_t(x)}{\sum_{x'} \tilde{b}_t(x')},$$

with $\varepsilon_b \in [10^{-12}, 10^{-8}]$.

Then the entropy is

$$H(b_t) = -\sum_{x \in \mathcal{X}} \bar{b}_t(x) \log \bar{b}_t(x).$$

4.7 What happens after detection and rescue

When a high-confidence target candidate is declared at $x^\star$, the system does *not* collapse the full map to a Kronecker delta immediately. Instead:

1. mark $x^\star$ as an active target hypothesis,
2. locally suppress redundant repeated information gain near already-confirmed cleared cells,
3. once a victim is physically confirmed and rescued, remove that victim from the unresolved target set,
4. update the mission-level unresolved victim map by zeroing or masking the resolved region and renormalizing remaining mass.

Thus entropy is computed over **unresolved victim uncertainty**, not over already rescued targets.

5 Why Shannon Entropy Is Used

For the exploration phase, uncertainty reduction is the natural objective. Shannon entropy

$$H(b_t) = -\sum_x b_t(x) \log b_t(x)$$

is used because it:

a) is maximal under diffuse uncertainty,

b) decreases when observations concentrate belief,

c) gives a scalar measure of mission information gain,

d) is independent of arbitrary target labels.

The exploration potential is

$$\Phi_E(s_t) = -H(b_t).$$

Important clarification. Entropy is the correct objective for `Explore`, but not for all mission stages. Once a target is detected, the mission transitions to motion and coordination objectives. Therefore this architecture uses **option-conditioned potentials**, not a single entropy-only potential for the entire task.

6 Option Set and Semantics

Each option is defined as

$$\omega = (I_\omega, \pi_\omega, \beta_\omega),$$

where I_ω is the initiation set, π_ω the intra-option policy, and β_ω the learned termination mechanism.

6.1 `Explore`

Intent: reduce uncertainty, expand coverage, avoid redundant revisits.

6.2 `Navigate`

Intent: move assigned agents toward a detected or selected target candidate.

6.3 `Form`

Intent: organize agents near the rescue site into a coordination geometry that supports a physical rescue primitive, safe standoff, or perimeter staging.

7 Option-Conditioned Potentials

The high-level reward is based on the option-conditioned potential:

$$\Phi(s, z) = \begin{cases} \Phi_E(s), & z = \texttt{Explore}, \\ \Phi_N(s), & z = \texttt{Navigate}, \\ \Phi_F(s), & z = \texttt{Form}. \end{cases}$$

7.1 Exploration potential

$$\Phi_E(s_t) = -H(b_t).$$

7.2 Navigation potential

Let g_t^i denote the target assigned to agent i during navigation. Then

$$d_i^{\text{goal}}(t) = \|p_t^i - g_t^i\|_2, \qquad \Phi_N(s_t) = -\sum_{i=1}^{N} d_i^{\text{goal}}(t).$$

7.3 Formation potential

Let $\mathcal{A}_t^{\text{form}}$ be the set of agents currently assigned to the formation zone around center c_t, and let

$$N_{\text{active}}(t) = |\mathcal{A}_t^{\text{form}}|.$$

The formation error is:

$$E_{\text{form}}(s_t) = \sum_{i \in \mathcal{A}_t^{\text{form}}} \left(\|p_t^i - c_t\|_2 - R_t\right)^2 + \lambda_\theta \sum_{i \in \mathcal{A}_t^{\text{form}}} \left(\Delta\theta_i - \frac{2\pi}{N_{\text{active}}(t)}\right)^2.$$

Then

$$\Phi_F(s_t) = -E_{\text{form}}(s_t).$$

8 Target Selection and Multi-Victim Allocation

8.1 Single-target baseline

In the simplest case, when only one unresolved target is considered, define:

$$g_t = \arg\max_x b_t(x), \qquad \text{valid only if } \max_x b_t(x) \geq \eta_{\text{det}}.$$

8.2 Multi-victim target extraction

In multi-victim mode, the training server extracts candidate targets from the unresolved victim map using local maxima or clustered peaks:

$$\mathcal{G}_t = \{g_t^{(1)}, \ldots, g_t^{(K_t)}\}.$$

These may be generated by:

1. non-maximum suppression over $b_t(x)$,
2. connected-component clustering over thresholded belief regions,
3. top-K MAP peak extraction.

8.3 Agent-to-target assignment

A navigation target g_t^i must be assigned per agent. This specification recommends one of the following:

Baseline assignment (simple). All navigating agents use the highest-confidence target:

$$g_t^i = g_t^{(1)} \quad \forall i.$$

This is easy to implement but not efficient for multi-victim missions.

Recommended assignment (multi-target). Let $\mathcal{I}_{\text{nav}}$ be the set of agents assigned to `Navigate`. Solve:

$$\min_\sigma \sum_{i \in \mathcal{I}_{\text{nav}}} \|p_t^i - g_t^{(\sigma(i))}\|_2,$$

subject to capacity constraints if needed. Hungarian assignment, greedy nearest-target assignment, or cluster-based partitioning are all acceptable.

This document recommends:

1. use the simple baseline for initial experiments,
2. use explicit target assignment for multi-victim evaluations,
3. report clearly which regime is used.

9 Graph Construction Under Limited Communication

9.1 Communication graph

At every timestep:

$$G_t = (V, E_t), \qquad V = \{1, \dots, N\},$$

with edges

$$(i, j) \in E_t \iff \|p_t^i - p_t^j\|_2 \le r_c,$$

where r_c is the communication radius.

This graph is the runtime communication substrate. It is what makes the approach decentralized and scalable: each agent exchanges information only with neighbors.

9.2 Node and edge features

The node feature for agent i is:

$$x_t^i = [o_t^i,\ z_k^i,\ p_t^i,\ v_t^i,\ m_t^{i,\mathrm{loc}}],$$

where:

- o_t^i = local observation,
- z_k^i = current active option embedding,
- p_t^i = local position/state representation,
- v_t^i = velocity or motion state,
- $m_t^{i,\mathrm{loc}}$ = optional compact local memory (e.g., previous worker hidden state).

This replaces the previously ambiguous ρ_t^i with explicit terms.

Edge feature:

$$e_t^{ij} = [p_t^j - p_t^i,\ \|p_t^j - p_t^i\|_2].$$

9.3 Worker message passing

A graph layer is defined as:

$$m_t^{ij} = \phi_\theta(h_t^i, h_t^j, e_t^{ij}), \qquad \bar{m}_t^i = \sum_{j \in \mathcal{N}_t(i)} m_t^{ij}, \qquad h_t^{i,+} = \psi_\theta(h_t^i, \bar{m}_t^i).$$

Scalability note. This makes message computation scale with $|E_t|$, not N^2. However, this is **structural scalability**. Performance scalability still requires empirical validation under larger N.

10 High-Level Manager

10.1 When the manager fires

The manager chooses an option for agent i only at option boundaries:

1. start of episode,
2. current option terminates,
3. hard timeout $k_{\max}$ is reached.

10.2 Manager input embedding

At each primitive timestep during option segment k, the worker computes a graph-updated hidden state h'^i_t.

At the next option boundary t_k, the manager history embedding is defined as:

$$h^i_{H,k} = \mathrm{LSTM}_H \left(h'^i_{t_{k-1}}, h'^i_{t_{k-1}+1}, \ldots, h'^i_{t_k} \right)_{\text{final}}.$$

Implementation decision. The sequence input to the manager is the sequence of worker-level graph embeddings over the most recent option segment. This is the default implementation. Raw observations are not fed directly to the manager LSTM in the base version.

10.3 Variable-length sequence handling

Because options have variable duration, training batches use padded sequences and masks, or packed sequences. The recommended implementation is:

a) store the sequence of h'^i_t values for each option segment,

b) pad to the maximum segment length within batch,

c) mask padded steps in the LSTM loss pipeline.

10.4 Manager output

$$z^i_k \sim \pi^i_H(\cdot \mid h^i_{H,k}), \qquad z^i_k \in \{\texttt{Explore}, \texttt{Navigate}, \texttt{Form}\}.$$

10.5 Why the manager does not use the full belief map at execution

The manager is decentralized at execution. It therefore cannot rely on a centralized belief map. Instead, training with global mission rewards teaches the manager to map local temporal patterns to useful option choices. This is an **empirical design assumption**, not a formal guarantee of optimality.

11 Low-Level Worker

11.1 Worker input

At every timestep:

$$h^i_{L,t} = \mathrm{GNN}_L \left(x^i_t, \{x^j_t\}_{j \in \mathcal{N}_t(i)} \right),$$

and the worker policy is:

$$a^i_t \sim \pi^i_L(\cdot \mid h^i_{L,t}, z^i_k).$$

Critical requirement. The active option z^i_k must be explicitly included in worker inputs. Otherwise the worker cannot distinguish exploration behavior from navigation or formation behavior.

11.2 Worker output

Primitive motion command:

$$a^i_t \in \mathcal{A}^i.$$

12 Initiation and Termination Sets

12.1 Thresholds

All threshold parameters are **global hyperparameters** shared across agents unless otherwise stated. Recommended initial ranges:

Parameter	Meaning / typical initial range
τ_E	exploration entropy threshold, e.g. 0.6–0.9 of initial entropy
η_{det}	detection confidence threshold, e.g. 0.6–0.8
τ_{conf}	confident low-entropy threshold, e.g. 0.2–0.4 of initial entropy
τ_N	navigation completion distance, e.g. 1–3 cells/meters
τ_F	formation activation radius, e.g. 2–5 cells/meters
τ_{form}	acceptable formation error, task-specific

12.2 Conflict precedence

If both

$$H(b) > \tau_E \qquad \text{and} \qquad \max_x b(x) \geq \eta_{\text{det}}$$

hold, then `Navigate` takes precedence over `Explore`. In other words, once a sufficiently confident target exists, search is no longer the primary mode.

12.3 `Explore` initiation and completion

$$I_E = \{\, s : H(b) > \tau_E,\ \max_x b(x) < \eta_{\text{det}} \,\}.$$

Completion set:

$$\mathcal{T}_E = \{\, s : \max_x b(x) \geq \eta_{\text{det}} \,\} \cup \{\, s : H(b) \leq \tau_{\text{conf}} \,\}.$$

12.4 `Navigate` initiation and completion

$$I_N = \{\, s : \exists g_t^i \text{ assigned and } \max_x b(x) \geq \eta_{\text{det}} \,\}.$$

Completion set:

$$\mathcal{T}_N = \left\{ s : \frac{1}{|\mathcal{I}_{\text{nav}}|} \sum_{i \in \mathcal{I}_{\text{nav}}} d_i^{\text{goal}} \leq \tau_N \right\}.$$

12.5 `Form` initiation and completion

$$I_F = \left\{ s : \frac{1}{N_{\text{active}}(t)} \sum_{i \in \mathcal{A}_t^{\text{form}}} \|p_t^i - c_t\|_2 \leq \tau_F \right\}.$$

Completion set:

$$\mathcal{T}_F = \{\, s : E_{\text{form}}(s) \leq \tau_{\text{form}} \,\} \cup \{\, s : \text{rescue primitive complete} \,\}.$$

13 Physical Justification of the Formation Option

The `Form` option is not merely aesthetic. It must correspond to a physically meaningful rescue coordination behavior. This document permits the following interpretations:

1. **Perimeter stabilization**: agents maintain safe standoff around a hazard or victim.
2. **Coordinated extraction staging**: multiple agents must occupy distributed positions around a target before a joint rescue primitive is enabled.
3. **Relay / containment geometry**: agents form a communication or sensing perimeter around a target.

If the application does not require ring-like geometry, replace the ring potential with a simpler *target-area occupancy potential*. The formation potential must be justified by the rescue physics of the chosen simulator or platform.

13.1 Formation radius

The formation radius R_t must be tied to task geometry. Recommended choice:

$$R_t = \max(r_{\text{safe}}, r_{\text{task}}),$$

where r_{safe} is a platform safety standoff and r_{task} is the radius required by the rescue primitive.

13.2 Which agent count should be used

Use:

$$N_{\text{active}}(t) = |\mathcal{A}_t^{\text{form}}|$$

in the angular spacing term. Use

$$N_{\text{required}}$$

only to define rescue completion logic, not geometric spacing. This resolves the earlier ambiguity.

14 Termination Module

14.1 Learned termination

After a minimum option duration $k_{\min}$,

$$\beta_t^i = \sigma\left(\text{MLP}\left(\text{GNN}_\beta(o_t^i, \{o_t^j\}_{j\in\mathcal{N}_t(i)})\right)\right).$$

14.2 Termination triggers

An option ends if:

1. the semantic completion set is met, or
2. the learned termination samples true after $k_{\min}$, or
3. the hard timeout $k_{\max}$ is reached.

14.3 Conflict resolution when no alternative option is valid

If learned termination fires but no alternative option is valid under the initiation sets, then:

1. keep the current option active for one additional step and re-evaluate, or
2. allow re-selection of the same option *without switch penalty.*

This avoids pathological option boundaries that force immediate self-reselection.

15 Training-Time Critics

15.1 High-level critic

At option boundary t_k:

$$\tilde{h}^i_{H,k} = \mathrm{GNN}_H(\tilde{x}^i_{t_k}, \{\tilde{x}^j_{t_k}\}_{j=1}^N), \qquad h^{\mathrm{glob}}_{H,k} = \mathrm{POOL}\big(\{\tilde{h}^i_{H,k}\}_{i=1}^N\big).$$

Then

$$V^H_\psi(s_{t_k}) = f^H_\psi(h^{\mathrm{glob}}_{H,k}).$$

15.2 Low-level critic

At every primitive step:

$$\tilde{h}^i_{L,t} = \mathrm{GNN}_{L,V}(\tilde{x}^i_t, \{\tilde{x}^j_t\}_{j=1}^N), \qquad h^{\mathrm{glob}}_{L,t} = \mathrm{POOL}\big(\{\tilde{h}^i_{L,t}\}_{i=1}^N\big).$$

Then

$$V^L_\psi(s_t) = f^L_\psi(h^{\mathrm{glob}}_{L,t}).$$

Execution note. Critics are not used during decentralized deployment.

16 Reward Design

16.1 High-level reward

For an option segment beginning at t_k and ending at t_{k+1}:

$$R^H_k = \Phi(s_{t_{k+1}}, z_k) - \Phi(s_{t_k}, z_k) - \lambda_t(t_{k+1} - t_k) - \lambda_{\mathrm{sw}}\mathbf{1}[z_k \neq z_{k-1}],$$

where switch penalty is omitted if forced self-reselection occurs under the termination fallback rule.

Expanded:

$$R^{H,E}_k = H(b_{t_k}) - H(b_{t_{k+1}}) - \lambda_t \Delta t - \lambda_{\mathrm{sw}}\mathbf{1}[\cdot],$$

$$R^{H,N}_k = \sum_{i=1}^N d^{\mathrm{goal}}_i(t_k) - \sum_{i=1}^N d^{\mathrm{goal}}_i(t_{k+1}) - \lambda_t \Delta t - \lambda_{\mathrm{sw}}\mathbf{1}[\cdot],$$

$$R^{H,F}_k = E_{\mathrm{form}}(t_k) - E_{\mathrm{form}}(t_{k+1}) - \lambda_t \Delta t - \lambda_{\mathrm{sw}}\mathbf{1}[\cdot].$$

16.2 Entropy exploit prevention

To avoid artificial entropy gains from repeatedly sweeping already-cleared cells, one or more of the following must be used:

1. **novelty mask**: compute information gain only over cells with uncertainty above threshold,
2. **minimum-gain threshold**:

$$\Delta H_{\text{eff}} = \max(H(b_{t_k}) - H(b_{t_{k+1}}) - \delta_H, 0),$$

3. **repeat-visit penalty**: penalize revisits to recently cleared cells,
4. **local coverage bonus**: reward previously unseen frontier expansion instead of raw negative observations.

16.3 Low-level worker reward

The worker uses a dense stepwise reward:

$$r_t^{L,i} = r_t^{\text{shared},i} + \alpha_z r_t^{\text{opt},i}(z_k^i) - \lambda_c \mathbf{1}[\text{collision}] - \lambda_u \mathbf{1}[\text{invalid move}].$$

16.4 Credit assignment-aware shared reward

To reduce the large-N credit assignment problem, do not assign rescue bonuses uniformly by default. Instead use:

$$r_t^{\text{shared},i} = w_t^i r_t^{\text{shared}}, \qquad w_t^i = \frac{\exp(-\|p_t^i - g_t^i\|_2/\sigma_w)}{\sum_j \exp(-\|p_t^j - g_t^j\|_2/\sigma_w)},$$

or a role-aware weighting that emphasizes agents near the relevant target. This is optional for the simplest baseline but recommended for larger teams.

16.5 Option-conditioned dense shaping

Recommended shaping:

$$r_t^{\text{opt},i}(\texttt{Explore}) = \Delta\text{local frontier coverage}_t^i - \lambda_r \mathbf{1}[\text{repeat visit}],$$

$$r_t^{\text{opt},i}(\texttt{Navigate}) = d_i^{\text{goal}}(t) - d_i^{\text{goal}}(t+1),$$

$$r_t^{\text{opt},i}(\texttt{Form}) = E_{\text{form}}^i(t) - E_{\text{form}}^i(t+1).$$

17 Mission-Specific Extension: Unknown Victim Count, Rescue Demand, and Parallel Subgroup Coordination

This section specializes the general SAR architecture to the following mission setting:

1. the number of victims is unknown a priori,
2. each victim v requires a victim-specific number k_v of agents for rescue,
3. k_v becomes observable only when an agent is sufficiently close to that victim,

4. agents assigned to one victim may rescue it while other agents continue exploring or respond to other detected victims,
5. rescue is completed when the required number of agents occupies valid rescue slots adjacent to the victim for a required hold duration.

17.1 Victim state representation

Let the set of all true victims in the environment be $\mathcal{V}^\star$, which is unknown to the agents. During execution and training, the system maintains a dynamic set of *detected victim tracks*

$$\mathcal{V}_{\text{det}}(t) = \{v_1, \ldots, v_{M_t}\},$$

where M_t is the number of currently detected but unresolved victims.

For each detected victim $v \in \mathcal{V}_{\text{det}}(t)$, define:

- estimated position p_v,
- rescue requirement k_v,
- current status
$$s_v(t) \in \{\texttt{detected}, \texttt{in_progress}, \texttt{rescued}\},$$
- currently assigned rescue-agent set $\mathcal{A}_v(t)$,
- nearby-agent count
$$n_v(t) = |\mathcal{A}_v(t)|.$$

Initially, no victim tracks exist:

$$\mathcal{V}_{\text{det}}(0) = \emptyset.$$

17.2 Unknown victim count

The number of victims is not known before execution. Therefore the mission does not terminate when the first victim is found or rescued. Instead, the system must continue exploration until the mission-complete condition is satisfied.

A practical mission-complete condition is:

$$\texttt{done} \iff \Big(\mathcal{V}_{\text{det}}(t) = \emptyset \text{ or all detected victims are rescued}\Big) \wedge \Big(\text{coverage criterion satisfied or time horizon reache}$$

A stronger version can maintain a frontier-completion or full-coverage criterion:

$$\texttt{done} \iff \Big(\forall v \in \mathcal{V}_{\text{det}}(t),\ s_v(t) = \texttt{rescued}\Big) \ \wedge\ \Big(\text{no significant unexplored region remains}\Big).$$

17.3 Victim discovery and reveal rule

A victim v becomes *detected* when at least one agent obtains sufficient evidence that a victim exists in a local region. Let r_{reveal} be the reveal radius. If

$$\|p_t^i - p_v\|_2 \le r_{\text{reveal}},$$

then agent i can observe victim-specific metadata, including the rescue requirement k_v.

Thus k_v is *not globally known* until some agent comes close enough. This is essential for the intended decentralized rescue-demand behavior.

17.4 Victim-demand signal

For each detected victim v, define its outstanding rescue demand:

$$d_v(t) = \max\big(0,\ k_v - n_v(t)\big).$$

This value indicates how many additional agents are still needed to complete rescue preparation around victim v.

The demand state can be broadcast only locally through graph message passing. Agents near victim v can include in their node/message features:

$$m_v^{\text{req}}(t) = \big[p_v,\ d_v(t),\ s_v(t)\big].$$

This means local messages are not merely generic coordination signals. They explicitly encode:

- victim location estimate,
- how many more agents are needed,
- whether rescue is already in progress or complete.

17.5 Parallel subgroup behavior

At any timestep, different subsets of agents may be executing different options:

$$z_t^i \in \{\texttt{Explore}, \texttt{Navigate}, \texttt{Form}\}.$$

Therefore it is valid for:

- one subgroup to be in `Form` around victim v_1,
- another subgroup to be in `Navigate` toward victim v_2,
- remaining agents to stay in `Explore`.

This is a central reason for using decentralized hierarchical control rather than a single centralized team action.

17.6 Agent assignment to victims

When multiple victims are detected simultaneously, each agent in `Navigate` or `Form` must be assigned to a victim. Let the set of unresolved detected victims be

$$\mathcal{V}_{\text{unres}}(t) = \{v \in \mathcal{V}_{\text{det}}(t) : s_v(t) \neq \texttt{rescued}\}.$$

A practical assignment rule is based on victim demand and travel distance:

$$J(i, v) = \lambda_d \|p_t^i - p_v\|_2 - \lambda_k d_v(t),$$

and assign each free agent i to the victim

$$v_i^\star = \arg\min_{v \in \mathcal{V}_{\text{unres}}(t)} J(i, v).$$

Interpretation:

- closer victims are cheaper,
- victims still needing more agents are prioritized.

For more precise assignment, a Hungarian matching or min-cost flow solver may be used.

17.7 Rescue slot formation

The earlier generic ring formation is replaced here by a *slot-based adjacent-cell rescue formation*, which is better matched to the mission statement.

For victim v at cell c_v, define the set of valid adjacent rescue slots:

$$\mathcal{S}_v = \{s_{v,1}, s_{v,2}, \ldots, s_{v,m_v}\},$$

where each slot is a valid free cell adjacent to c_v under the map geometry. Usually,

$$m_v \leq 8$$

for a grid with Moore adjacency, minus blocked cells.

If victim v requires k_v agents, the rescue controller selects a subset

$$\mathcal{S}_v^{(k)} \subseteq \mathcal{S}_v, \qquad |\mathcal{S}_v^{(k)}| = k_v.$$

The active rescue agents for victim v should occupy these slots. The slot-assignment formation error is:

$$E_{\text{form}}^v(t) = \min_{\sigma} \sum_{j=1}^{k_v} \|p_t^{\sigma(j)} - s_{v,j}\|_2^2,$$

where σ assigns the selected rescue agents in $\mathcal{A}_v(t)$ to the chosen rescue slots.

This replaces the earlier ring spacing objective for missions in which the physically meaningful rescue geometry is adjacency-based.

17.8 Formation initiation rule

The `Form` option for victim v becomes valid only if:

a) victim v has been detected,

b) k_v is known,

c) enough agents are assigned or sufficiently close:

$$n_v(t) \geq k_v \quad \text{or} \quad |\{i : \|p_t^i - p_v\|_2 \leq \tau_F\}| \geq k_v,$$

d) the rescue slots are feasible.

Thus `Form` means:

> "occupy the required adjacent rescue slots around the victim"

rather than an abstract ring arrangement.

17.9 Formation completion and rescue completion

Define the occupancy indicator for victim v:

$$\chi_v(t) = \mathbf{1}\big[\text{all required rescue slots for } v \text{ are occupied}\big].$$

To avoid one-step accidental completion, require the slots to be held for a minimum dwell time T_{hold}:

$$\sum_{\tau=t-T_{\text{hold}}+1}^{t} \chi_v(\tau) = T_{\text{hold}}.$$

Then rescue completion is:

$$s_v(t) = \texttt{rescued} \quad \text{if and only if} \quad \chi_v(\tau) = 1 \text{ for } T_{\text{hold}} \text{ consecutive steps.}$$

Once victim v is rescued:

1. remove v from the unresolved target set,
2. set
$$d_v(t) = 0,$$
3. clear victim-specific local request messages,
4. release agents in $\mathcal{A}_v(t)$ back to the high-level manager.

17.10 Post-rescue option reassignment

After rescue completion of victim v, each released agent immediately returns to an option boundary and reselects its next behavior:

$$z^i_{k+1} \sim \pi^i_H(\cdot \mid h^i_{H,k+1}).$$

The preferred fallback logic is:

a) if another unresolved victim with positive demand is known nearby, choose `Navigate`,

b) otherwise choose `Explore`.

This produces the intended loop:

$$\texttt{Explore} \to \texttt{Navigate} \to \texttt{Form} \to \texttt{Explore}$$

for one rescued subgroup, while other subgroups may continue independent rescue chains.

17.11 Mission-specific navigation potential

For this mission, navigation should be demand-aware. Instead of rewarding only proximity to a victim, it is useful to reward movement toward an *assigned unresolved victim* with positive demand:

$$\Phi_N(s_t) = -\sum_{i \in \mathcal{I}_{\text{nav}}} \|p^i_t - p_{v^\star_i}\|_2,$$

where $v^\star_i$ is the assigned victim for agent i.

This prevents the entire team from collapsing onto the same victim if other victims also require help.

17.12 Mission-specific formation potential

For this mission, replace the generic formation potential by:

$$\Phi_F(s_t) = -\sum_{v \in \mathcal{V}_{\text{in-progress}}(t)} E^v_{\text{form}}(t),$$

where $\mathcal{V}_{\text{in-progress}}(t)$ is the set of detected victims currently under active rescue formation.

17.13 Mission-specific low-level rewards

The worker reward should reflect the mission phase:

$$r^{L,i}_t(\texttt{Explore}) = \Delta\text{frontier coverage}^i_t - \lambda_r \mathbf{1}[\text{repeat visit}],$$

$$r^{L,i}_t(\texttt{Navigate}) = \|p^i_t - p_{v^\star_i}\|_2 - \|p^i_{t+1} - p_{v^\star_i}\|_2,$$

$$r^{L,i}_t(\texttt{Form}) = E^{v^\star_i,i}_{\text{form}}(t) - E^{v^\star_i,i}_{\text{form}}(t+1),$$

plus any shared rescue-completion bonus.

17.14 Mission-specific high-level reward

The manager reward remains option-conditioned, but now with mission-specific victim states:

- `Explore`: entropy reduction over unresolved victim uncertainty,
- `Navigate`: reduction in assigned-victim distances,
- `Form`: reduction in slot-assignment formation error,
- rescue-completion bonus when a victim transitions to `rescued`.

A useful additional term is:

$$R^{H,\text{rescue}}_k = \lambda_{\text{rescue}} \cdot \#\{v : s_v(t_{k+1}) = \texttt{rescued} \text{ and } s_v(t_k) \neq \texttt{rescued}\}.$$

17.15 Execution summary for this mission

With the above extension, the intended runtime behavior is:

1. all agents begin in `Explore`,
2. when a victim is locally discovered, nearby agents reveal k_v,
3. local message passing propagates demand $d_v(t)$,
4. enough free agents switch to `Navigate` toward that victim,
5. once k_v agents arrive, they switch to `Form` and occupy adjacent rescue slots,
6. rescue completes after slot occupancy is held for T_{hold},
7. rescued agents are released and rejoin exploration or another pending rescue,
8. other agents continue exploring or servicing different victims in parallel.

This mission-specific extension is the recommended formulation for unknown-victim-count decentralized SAR with variable rescue demand and local recruitment.

18 Optimization and Learning

18.1 High-level advantage and PPO

$$A_k^H = G_k^H - V_\psi^H(s_{t_k}).$$

Manager objective:

$$L_H = \mathbb{E}\left[\min\left(r_k^H(\theta_H)A_k^H, \text{clip}(r_k^H(\theta_H), 1-\epsilon_H, 1+\epsilon_H)A_k^H\right)\right],$$

with

$$r_k^H(\theta_H) = \frac{\pi_H(z_k \mid h_{H,k})}{\pi_H^{\text{old}}(z_k \mid h_{H,k})}.$$

18.2 Low-level advantage and MAPPO

$$A_t^L = G_t^L - V_\psi^L(s_t).$$

Worker objective:

$$L_L = \mathbb{E}\left[\min\left(r_t^L(\theta_L)A_t^L, \text{clip}(r_t^L(\theta_L), 1-\epsilon_L, 1+\epsilon_L)A_t^L\right)\right].$$

18.3 Why PPO for manager and MAPPO for worker

The worker operates at the primitive-action timescale where inter-agent coupling is strongest. Therefore MAPPO with centralized low-level critics is appropriate. The manager operates at sparse option boundaries over macro-actions, so PPO with a centralized high-level critic is sufficient and simpler at the option timescale. This asymmetry is a design choice motivated by timescale separation and implementation stability, not by a theoretical optimality result.

18.4 Termination loss

All primitive steps within option segment k share the same macro-advantage A_k^H. The termination loss is:

$$\mathcal{L}_\beta = \mathbb{E}\left[\beta_t^i A_k^H\right] - \lambda_\beta \mathcal{H}(\beta_t^i) + \lambda_d \mathcal{L}_{\text{dur}},$$

where:

- $\mathcal{H}(\beta_t^i)$ is Bernoulli entropy regularization,
- $\mathcal{L}_{\text{dur}}$ penalizes pathological very short or very long option durations.

19 Replay / Rollout Storage Requirements

Because option boundaries are variable, the high-level reward cannot be computed until an option terminates. Therefore the rollout buffer must store:

1. boundary index t_k,
2. option label z_k^i,
3. manager input $h_{H,k}^i$,
4. belief snapshot statistics needed at t_k,

5. primitive transitions accumulated until t_{k+1},
6. boundary belief statistics at t_{k+1}.

It is sufficient to store the scalar values needed for reward (e.g., entropy, goal distances, formation error) rather than the full map in every manager tuple, unless diagnostics require the full belief snapshot.

20 Training Curriculum

Hierarchical systems are unstable if both levels start fully random. The recommended schedule is:

20.1 Phase 1: low-level pretraining

Freeze the manager and train workers separately with fixed option labels.

a) `Explore`: frontier/coverage reward.

b) `Navigate`: distance-to-goal reward using scripted targets.

c) `Form`: formation reward around scripted centers.

20.2 Phase 2: termination warm-start

Train or initialize the termination network with semantic labels derived from option completion conditions, or keep it frozen until workers become competent.

20.3 Phase 3: joint training

Unfreeze the manager and perform full hierarchical training with:

- manager PPO,
- worker MAPPO,
- centralized critics,
- learned termination.

20.4 Phase 4: fine-tuning

Reduce learning rates and stabilize:

- switch penalty,
- termination entropy regularization,
- credit-assignment weighting.

21 Execution Algorithm

1. Reset environment.
2. Initialize or update training-side unresolved victim belief map.
3. For each agent, if at option boundary, compute $h^i_{H,k}$ and select valid option z^i_k.
4. Build local communication graph G_t.
5. Compute worker graph embeddings $h^i_{L,t}$.
6. Sample primitive actions a^i_t.
7. Step environment.
8. On training server, update belief map using joint observation model.
9. Evaluate semantic completion and learned termination.
10. If option terminates, compute macro reward retroactively.
11. Continue until rescue complete or horizon reached.

22 Decentralization Claim: What Is and Is Not Guaranteed

This architecture is decentralized by construction at execution time because agents use only:

- local observations,
- local graph messages,
- local memory,
- current option label.

However, there is **no formal theorem** in this document guaranteeing that a policy trained with global belief-driven rewards will achieve near-oracle performance under local observations only. That claim is empirical and must be validated experimentally against stronger-information baselines.

23 Convergence and Stability Claim

This document does not claim a formal convergence guarantee for the simultaneous training of:

- high-level PPO,
- low-level MAPPO,
- learned termination,
- two centralized critics.

Instead, stability is pursued through:

1. worker pretraining,
2. clipped PPO objectives,
3. minimum/maximum option duration,
4. entropy-regularized termination,
5. delayed joint fine-tuning.

24 Scalability Claim

There are two distinct notions of scalability:

24.1 Structural scalability

Graph message passing uses shared parameters independent of team size, and runtime communication is local. This is a design property.

24.2 Performance scalability

Whether a policy trained at one team size generalizes to larger team sizes is an empirical question because neighborhood statistics and message distributions shift as N grows.

Therefore this specification recommends explicit experiments over:

$$N \in \{5, 10, 20, 40\}$$

with train-test transfer protocols such as:

- train on $N = 5$, test on $N = 10, 20, 40$,
- vary communication radius r_c,
- vary victim count and target density.

25 Recommended Metrics

Track:

1. entropy reduction rate,
2. time to first detection,
3. time to full rescue,
4. redundant coverage overlap,
5. average option duration,
6. option-switch frequency,
7. bottleneck throughput,
8. success rate vs. team size,
9. performance under reduced communication radius.

26 Summary of Final Design Decisions

1. Belief map is centrally maintained during training using a joint Bayesian update.
2. Initial belief is uniform unless domain priors exist.
3. Shannon entropy drives only the `Explore` option.
4. Option-conditioned potentials define macro rewards.
5. Multi-victim mode requires explicit target extraction and assignment.
6. Worker uses local observation, explicit option label, motion state, and local graph messages.
7. Manager uses an LSTM over worker graph embeddings from the previous option segment.
8. Termination uses semantic completion, learned β, and timeout with fallback when no valid alternative exists.
9. Formation geometry uses N_{active}, not total N.
10. Worker pretraining is recommended before joint hierarchical training.
11. Credit assignment at scale should use proximity- or role-weighted shared rewards.
12. Structural scalability does not imply performance scalability; transfer must be tested.